

Final Exam Preparation: Level 5 Solutions

Reference: *LEVEL 5.pdf*

Below are the correct C solutions for the advanced Level 5 exercises. All these problems require full program implementations (including `main`) to handle complex parsing, stack logic, or state management.

1. brainfuck

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        char tape[2048] = {0};
        int ptr = 0;
        int i = 0;
        int loop;
        char *code = argv[1];

        while (code[i])
        {
            if (code[i] == '>') ptr++;
            else if (code[i] == '<') ptr--;
            else if (code[i] == '+') tape[ptr]++;
            else if (code[i] == '-') tape[ptr]--;
            else if (code[i] == '.') write(1, &tape[ptr], 1);
            else if (code[i] == '[' && tape[ptr] == 0)
            {
                loop = 1;
                while (loop > 0)
                {
                    i++;
                    if (code[i] == '[') loop++;
                    if (code[i] == ']') loop--;
                }
            }
            i++;
        }
    }
}
```

```
        else if (code[i] == ']' && tape[ptr] != 0)
        {
            loop = 1;
            while (loop > 0)
            {
                i--;
                if (code[i] == ']') loop++;
                if (code[i] == '[') loop--;
            }
        }
        i++;
    }
}
write(1, "\n", 1);
return (0);
}
```

2. brackets

```
#include <unistd.h>

int check_brackets(char *str)
{
    char stack[4096];
    int top = 0;
    int i = 0;

    while (str[i])
    {
        if (str[i] == '(' || str[i] == '[' || str[i] == '{')
        {
            stack[top++] = str[i];
        }
        else if (str[i] == ')' || str[i] == ']' || str[i] == '}')
        {
            if (top == 0)
                return (0);
            top--;
            if (str[i] == ')') && stack[top] != '(') return (0);
        }
    }
}
```

```
        if (str[i] == ']' && stack[top] != '[') return (0);
        if (str[i] == '}' && stack[top] != '{') return (0);
    }
    i++;
}
return (top == 0);
}

int main(int argc, char **argv)
{
    if (argc > 1)
    {
        int i = 1;
        while (i < argc)
        {
            if (check_brackets(argv[i]))
                write(1, "OK\n", 3);
            else
                write(1, "Error\n", 6);
            i++;
        }
    }
    else
        write(1, "\n", 1);
    return (0);
}
```

3. check_mate

```
#include <unistd.h>

int ft_strlen(char *s)
{
    int i = 0;
    while (s[i])
        i++;
    return (i);
}
```

```
int check_dir(char **board, int size, int r, int c, int dr, int dc)
{
    int i = r + dr;
    int j = c + dc;
    int is_pawn_range = 1;

    while (i >= 0 && i < size && j >= 0 && j < size)
    {
        char p = board[i][j];
        if (p != '.')
        {
            if (dr != 0 && dc != 0) // Checking Diagonals
            {
                if (p == 'B' || p == 'Q')
                    return (1);
                // Pawns attack downwards, so from King's perspective
                we look UP (dr == -1)
                if (p == 'P' && dr == -1 && is_pawn_range)
                    return (1);
            }
            else // Checking Straights (Horizontal/Vertical)
            {
                if (p == 'R' || p == 'Q')
                    return (1);
            }
            return (0); // Sightline blocked by another piece
        }
        is_pawn_range = 0;
        i += dr;
        j += dc;
    }
    return (0);
}

int main(int argc, char **argv)
{
    if (argc > 1)
    {
        int size = argc - 1;
        int kr = -1, kc = -1, kings = 0;
```

```
for (int i = 0; i < size; i++)
{
    if (ft_strlen(argv[i + 1]) != size)
    {
        write(1, "\n", 1);
        return (0);
    }
    for (int j = 0; j < size; j++)
    {
        if (argv[i + 1][j] == 'K')
        {
            kings++;
            kr = i;
            kc = j;
        }
    }
}

if (kings != 1)
{
    write(1, "\n", 1);
    return (0);
}

char **b = &argv[1];
if (check_dir(b, size, kr, kc, -1, -1) || check_dir(b, size,
kr, kc, -1, 1) ||
    check_dir(b, size, kr, kc, 1, -1) || check_dir(b, size,
kr, kc, 1, 1) ||
    check_dir(b, size, kr, kc, -1, 0) || check_dir(b, size,
kr, kc, 1, 0) ||
    check_dir(b, size, kr, kc, 0, -1) || check_dir(b, size,
kr, kc, 0, 1))
    write(1, "Success\n", 8);
else
    write(1, "Fail\n", 5);
}
else
    write(1, "\n", 1);
```

```
    return (0);  
}
```

4. options

```
#include <unistd.h>  
  
int main(int argc, char **argv)  
{  
    if (argc == 1)  
    {  
        write(1, "options: abcdefghijklmnopqrstuvwxyz\n", 36);  
        return (0);  
    }  
  
    int opt = 0;  
    int i = 1;  
  
    while (i < argc)  
    {  
        if (argv[i][0] == '-')  
        {  
            int j = 1;  
            while (argv[i][j])  
            {  
                if (argv[i][j] == 'h')  
                {  
                    write(1, "options: abcdefghijklmnopqrstuvwxyz\n",  
36);  
                    return (0);  
                }  
                if (argv[i][j] < 'a' || argv[i][j] > 'z')  
                {  
                    write(1, "Invalid Option\n", 15);  
                    return (0);  
                }  
                // Toggle bit on (using 1 shifted by the letter's  
index)
```

```
        opt |= (1 << (argv[i][j] - 'a'));
        j++;
    }
}
i++;
}

// Print bits starting from the 32nd bit down to the 1st
i = 31;
while (i >= 0)
{
    if (opt & (1 << i))
        write(1, "1", 1);
    else
        write(1, "0", 1);

    // Add a space after every 8 bits (except at the very end)
    if (i % 8 == 0 && i != 0)
        write(1, " ", 1);
    i--;
}
write(1, "\n", 1);

return (0);
}
```

5. rpn_calc

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    if (argc != 2)
    {
        printf("Error\n");
        return (0);
    }
}
```

```
char *s = argv[1];
int stack[4096];
int top = 0;
int i = 0;

while (s[i])
{
    // Skip whitespace
    if (s[i] == ' ' || s[i] == '\t')
    {
        i++;
        continue;
    }

    // Handle numbers (including negative numbers)
    if ((s[i] >= '0' && s[i] <= '9') ||
        (s[i] == '-' && s[i + 1] >= '0' && s[i + 1] <= '9'))
    {
        stack[top++] = atoi(&s[i]);
        if (s[i] == '-') i++;
        while (s[i] >= '0' && s[i] <= '9') i++;
    }

    // Handle operators
    else if (s[i] == '+' || s[i] == '-' || s[i] == '*' ||
             s[i] == '/' || s[i] == '%')
    {
        if (top < 2)
        {
            printf("Error\n");
            return (0);
        }
        int b = stack[--top];
        int a = stack[--top];

        if (s[i] == '+') stack[top++] = a + b;
        else if (s[i] == '-') stack[top++] = a - b;
        else if (s[i] == '*') stack[top++] = a * b;
        else if (s[i] == '/')
        {
            if (b == 0) { printf("Error\n"); return (0); }
```



```
        stack[top++] = a / b;
    }
    else if (s[i] == '%')
    {
        if (b == 0) { printf("Error\n"); return (0); }
        stack[top++] = a % b;
    }
    i++;
}
else // Invalid characters detected
{
    printf("Error\n");
    return (0);
}
}

if (top == 1)
    printf("%d\n", stack[0]);
else
    printf("Error\n");

return (0);
}
```